Energy-Aware Priority Scheduling in Smart Sensor Networks Using Policy-Guided Deep Reinforcement Learning

Anand Emmanuel Raju B.[1], Radhika N.[1], and Radhika G.[1]

Department of Computer Science and Engineering,
Amrita School of Computing, Coimbatore,
Amrita Vishwa Vidyapeetham, India
cb.sc.p2cse25003@cb.students.amrita.edu, {n_radhika, g_radhika}@cb.amrita.edu

Abstract. This paper explores deep reinforcement learning (DRL) for packet-level queue scheduling within energy-constrained wireless sensor networks (WSNs) carrying heterogeneous traffic profiles—specifically, urgent alarm signals alongside routine telemetry. Interfacing PyTorch with the ns-3 simulator via the ns3-gym middleware, we train a Double-DQN scheduler enhanced by prioritized experience replay. To mitigate priority starvation and policy instability, we formulate a soft, state-dependent reward guidance mechanism. Across 20 seed-synchronized, independent co-simulation trials, we analyze this learned controller against a deterministic Strict Priority baseline and a uniform random scheduler. Although our reinforcement learning agent easily outperforms random scheduling and preserves high delivery rates for priority traffic, it falls short of the Strict Priority baseline in throughput, overall packet delivery ratio, and high-priority delay. We report these findings as a transparent, empirical benchmark that defines a clear performance boundary for model-free controllers in safety-critical sensor networks.

Keywords: Smart Energy Systems · Wireless Sensor Networks · Reinforcement Learning · ns-3 · QoS · Priority Scheduling

1 Introduction

Modern smart energy infrastructures rely heavily on Wireless Sensor Networks (WSNs) deployed at the edge to monitor grid stability, track consumption patterns, and report critical fault events. Operating under severe energy limitations, these sensor nodes frequently handle mixed-criticality traffic workloads. For instance, routine periodic telemetry (normal-priority traffic) must coexist with sporadic, time-sensitive emergency alerts (high-priority traffic) triggered by system anomalies or grid faults. Under bursty load conditions, the underlying packet scheduling mechanism at each sensor node directly dictates critical network performance metrics, including the packet delivery ratio (PDR), deadline compliance, end-to-end latency, and the overall operational lifetime of the battery-powered nodes [4,7].

In smart grid applications, latency and reliability are non-negotiable. Delayed delivery of a fault alert can prevent protective relays from isolating a short circuit, leading to catastrophic equipment damage or cascading grid failures. Traditional static queue scheduling heuristics, such as Strict Priority (SP), are widely used due to their simple design and predictable execution. However, static rules are inherently rigid and cannot adapt to dynamic network conditions, varying traffic intensities, or changing residual energy profiles. For example, Strict Priority blindly prioritizes high-priority packets, which often leads to severe queue starvation, high buffer overflows, and excessive drop rates for normal telemetry data during prolonged congestion periods.

To address these limitations, Deep Reinforcement Learning (DRL) has emerged as a promising paradigm for learning adaptive, state-aware control policies directly from closed-loop interaction with the environment. By modeling queue scheduling as a Markov Decision Process (MDP), a DRL agent can theoretically discover optimal scheduling policies that balance conflicting network objectives. However, deploying model-free DRL agents in actual communication systems introduces significant challenges, including slow convergence, extreme policy instability during early training phases, and the risk of policy collapse. In multi-objective networking scenarios, a poorly constrained reward function often leads the agent to discover degenerate local optima, such as completely ignoring high-priority packets to maximize the delivery of more frequent normal-priority traffic.

This paper presents a systematic, empirical evaluation of a policy-guided Double Deep Q-Network (Double-DQN) queue scheduler using an integrated co-simulation framework that couples PyTorch with the ns-3 network simulator via the ns3-gym interface [5,2]. Rather than presenting an idealized, overstated success story, we investigate a narrower, more rigorous systems question: can a learned DRL policy reliably maintain high-priority service quality while offering stable, competitive performance against a highly optimized Strict Priority baseline under repeatable, seed-controlled evaluation conditions?

The core contributions of this work are summarized as follows:

- We establish a fully open-source, reproducible end-to-end co-simulation pipeline that links the ns-3 network simulator with Python-based DRL libraries, enabling run-level artifact verification.
- We design a soft, state-dependent reward guidance mechanism and an aligned evaluation score that explicitly tracks high-priority packet generation, delivery, and age, preventing the agent from collapsing into degenerate, non-adaptive scheduling behaviors.
- We perform a statistically rigorous comparison of our learned DQN controller against both Strict Priority and Random scheduling baselines over 20 independent, seed-synchronized evaluation runs.
- We present a transparent empirical benchmark, demonstrating that while the learned agent achieves excellent normal-traffic latency improvements and easily outperforms random baselines, the classic Strict Priority heuristic

remains highly dominant on primary high-priority QoS metrics under the tested workloads.

2 Related Work

Recent academic literature extensively documents the application of reinforcement learning and deep reinforcement learning to wireless communication networks, spanning medium access control (MAC), routing, and packet scheduling [8,4]. While these surveys highlight the massive theoretical potential of learning-based controllers, they consistently point to critical barriers preventing real-world deployment, namely sample inefficiency, lack of safety or priority guarantees, and poor generalization under out-of-distribution traffic patterns.

In WSNs, DRL has been actively explored to optimize sensor scheduling, duty cycles, and energy-aware routing protocols. Leong *et al.* investigated deep reinforcement learning for wireless sensor scheduling in cyber-physical systems, demonstrating that model-free controllers can successfully adapt to dynamic channel uncertainties [7]. Sharma and Chauhan proposed a distributed reinforcement learning scheduling algorithm designed to maintain coverage and connectivity constraints while minimizing energy consumption [11]. Similarly, Dutta and Biswas developed a distributed RL framework to optimize medium access control in dense IoT sensor deployments [3], while Barker and Swany explored cooperative multi-agent RL to solve complex routing problems in multi-hop WSNs [1].

More specialized studies have focused on aligning RL objectives with physical hardware constraints, such as sleep-scheduling and safety bounds. Mishra and Liang presented an RL-driven dynamic sleep scheduling policy to maximize sensor lifetime by balancing active transmission states and low-power sleep states [9]. Nagib *et al.* proposed safe DRL design methodologies, emphasizing the use of action projection and reward shaping to enforce hard operational constraints in next-generation wireless systems [10]. Additionally, Kim *et al.* demonstrated the feasibility of using DRL-based adaptive schedulers to meet stringent latency requirements in industrial wireless time-sensitive networking (TSN) environments [6].

In contrast to existing works that primarily introduce minor algorithmic variations, this paper focuses on systems-level execution and empirical transparency. We provide a fully reproducible benchmark study with explicit high-priority service instrumentation, validation-gated model selection, and run-level statistical verification to define clear performance boundaries for DRL-based schedulers in mixed-criticality sensor networks.

3 System Model and Problem Formulation

3.1 Network and Traffic Model

We consider a wireless sensor network scenario consisting of 100 sensor nodes distributed dynamically around a central sink node, simulated using the discrete-

event network simulator ns-3. The sensor nodes generate two distinct classes of traffic: High-Priority (HP) emergency packets representing critical grid alerts, and Normal-Priority (NP) periodic telemetry packets. Each node maintains separate, finite-capacity buffers for the two traffic classes. The scheduling agent is located at the edge (e.g., at a cluster head or local gateway) and operates at a discrete control interval of 100 ms. Each simulation episode runs for exactly 5000 decision steps. Crucially, the simulator tracks the generation, delivery, queue occupancy, latency, and timeout/drop events individually for both packet classes, exporting these metrics at the end of each episode to provide complete visibility.

3.2 MDP Definition

At each discrete decision step t, the gateway scheduling agent observes the network state and selects an action. We model this scheduling process as a finite Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

The state space $\mathcal{S}$ is represented by a 6-dimensional normalized vector:

$$\mathbf{s}_t = [q_t^N,\ q_t^H,\ e_t,\ a_t^N,\ a_t^H,\ d_t], \tag{1}$$

where q_t^N and q_t^H represent the normalized occupancy proxies of the normal and high-priority queues, respectively; e_t denotes the residual energy proxy of the gateway node; a_t^N and a_t^H represent the normalized ages of the oldest packets waiting at the head of each queue; and d_t represents the local neighborhood node density proxy, reflecting dynamic channel contention.

The action space $\mathcal{A}$ consists of 5 discrete scheduling choices:

$$\mathcal{A} = \{\text{send HP},\ \text{send normal},\ \text{drop normal},\ \text{sleep},\ \text{drop HP}\}. \tag{2}$$

Selecting "send HP" or "send normal" initiates the transmission of a packet from the respective queue. "Sleep" puts the radio transceiver into a low-power state for the current interval to conserve residual energy, while "drop normal" and "drop HP" allow the agent to actively clear buffer space to prevent uncontrolled buffer bloat during extreme congestion.

3.3 Reward Design

Enforcing priority queueing behavior in model-free agents is notoriously difficult. If the agent receives a simple linear reward based only on total packet delivery, it will naturally prioritize the normal queue because normal packets are generated much more frequently, leading to policy collapse. Conversely, if the reward is too heavily biased towards high-priority packets, the agent will learn a rigid policy that blindly mimics a basic Strict Priority heuristic, destroying any adaptive capability.

To resolve this objective conflict, we design an outcome-driven, soft-guided reward function. The total step reward r_t is defined as:

$$r_t = \tanh\left(\frac{r_t^{\text{base}} + r_t^{\text{guide}}}{100}\right), \tag{3}$$

where the hyperbolic tangent function scales the reward to the range $[-1, 1]$ to stabilize neural network gradient updates. The term r_t^{base} is the basic reward reflecting standard network performance:

$$r_t^{\text{base}} = w_1 \cdot \text{HP}_{\text{sent}} + w_2 \cdot \text{Normal}_{\text{sent}} + w_3 \cdot \text{HP}_{\text{dropped}} + w_4 \cdot \text{Normal}_{\text{dropped}} + w_5 \cdot (a_t^H)^2 + w_6 \cdot (a_t^N)^2 - w_7 \cdot \Delta e_t, \tag{4}$$

where HP_{sent} and $\text{Normal}_{\text{sent}}$ are binary indicators of successful packet transmissions; $\text{HP}_{\text{dropped}}$ and $\text{Normal}_{\text{dropped}}$ penalize buffer drops and timeouts; $(a_t^H)^2$ and $(a_t^N)^2$ apply quadratic penalties to waiting packet ages to minimize latency; and Δe_t penalizes energy consumption.

The key novel component is the state-dependent, soft policy guidance term r_t^{guide}, defined as:

$$r_t^{\text{guide}} = \begin{cases} +\alpha, & a_t = \text{send HP and } q_t^H > 0 \\ -\beta, & a_t \neq \text{send HP},\ q_t^H > 0,\ a_t^H > \tau_u \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

Here, $\alpha = 6.0$ provides a moderate bonus for actively serving the high-priority queue when packets are waiting. Crucially, the penalty $\beta = -8.0$ is only applied for ignoring the high-priority queue if the oldest high-priority packet has aged beyond an urgency threshold $\tau_u = 0.55$. This "soft guidance" formulation ensures that the agent is not penalized for temporarily delaying newly arrived high-priority packets to drain a congested normal buffer, preserving policy flexibility while strictly preventing high-priority packet starvation.

4 Methodology

4.1 Learning Agent

To find the optimal scheduling policy, we implement a Double Deep Q-Network (Double-DQN) agent. Standard DQN is highly susceptible to overestimating action-values (Q-values) due to the maximization step in the temporal difference target. Double-DQN decouples the action selection from the action evaluation by using the online network to select the best action and the target network to evaluate its Q-value:

$$Y_t^{\text{DoubleQ}} = r_{t+1} + \gamma Q\left(s_{t+1}, \arg\max_a Q(s_{t+1}, a; \theta_t); \theta_t^-\right), \tag{6}$$

where θ_t represents the parameters of the online network, and θ_t^- represents the parameters of the target network.

To further improve sample efficiency and training stability, we incorporate a Prioritized Replay Buffer (PER). Instead of sampling transitions uniformly from the replay memory, PER transitions are sampled with a probability proportional to their temporal difference (TD) error, ensuring the agent focuses learning on states with high unexpected transitions. The target network is updated using soft target updates:

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-, \tag{7}$$

with $\tau = 0.005$, ensuring smooth policy transitions. The neural network architecture consists of an input layer, two fully connected hidden layers with 64 units each and ReLU activation functions, and an output layer matching the action space size. We use the Adam optimizer with an initial learning rate of 1×10^{-4} and apply a Cosine Annealing learning rate schedule over the training epochs.

During training, the agent balances exploration and exploitation using an ϵ-greedy policy, where ϵ decays exponentially from 1.0 to a minimum floor of 0.05. During the evaluation phase, ϵ is set to exactly 0.0 to ensure deterministic policy execution and eliminate exploration-induced noise.

4.2 Baselines

We compare the performance of our proposed DQN-Edge agent against two baseline scheduling policies:

- **Strict Priority (SP)**: A hardcoded heuristic that always transmits high-priority packets if any are present in the HP buffer ($q_t^H > 0$). If the HP buffer is empty, it transmits normal-priority packets ($q_t^N > 0$). If both buffers are empty, the node enters the low-power sleep state.
- **Random**: A baseline where the agent selects one of the five actions uniformly at random at each decision step, serving as a lower bound on network performance.

4.3 Training and Evaluation Protocol

To ensure scientific rigor, we employ a seed-controlled evaluation protocol. The training process runs for 50 epochs, with 5000 decision steps per epoch. At the end of each epoch, the current policy is validated across five independent validation seeds. The model that achieves the highest mean validation score is saved as the optimal policy.

The validation score is specifically designed to prevent the saving of collapsed policies. It evaluates the agent on four key dimensions:

$$\text{Score} = 0.40 \cdot \text{PDR} + 0.10 \cdot (1 - \text{PLR}) + 0.35 \cdot \text{HP}_{\text{Ratio}} + 0.15 \cdot \text{HP}_{\text{Delay\_Comp}}, \quad (8)$$

where HP_{Ratio} is the high-priority delivery ratio, and $\text{HP}_{\text{Delay\_Comp}}$ is a normalized penalty component based on high-priority packet latency. Crucially, we apply a severe penalty of -0.25 to the validation score if high-priority packets were generated during the episode but the agent failed to deliver a single one. This explicitly prevents saving "lazy" policies that maximize overall delivery by completely starving priority traffic.

After training, the saved optimal policy is evaluated across 20 independent, seed-synchronized runs. These 20 seeds are identical across all three evaluated policies (DQN-Edge, Strict Priority, and Random), ensuring a perfectly fair, matched comparison.

Table 1. Simulation and training configuration (V76).

Category	Value
Simulator	ns-3.40 with ns3-gym interface
Nodes / sink	100 nodes + 1 sink
Control interval	100 ms
Episode length	5000 steps
Training epochs	50 (early stopping patience: 15)
Evaluation runs	20 per policy
State / action size	6 / 5
Replay buffer	100,000 (prioritized replay)
Mini-batch size	128
Warm-up steps	4000
Discount factor γ	0.99
Optimizer	Adam (1×10^{-4} initial LR)
Target update	Soft update ($\tau = 0.005$)
Exploration	ϵ: 1.0 $\rightarrow$ 0.05, decay 0.99995
Random seeds	Global base seed + fixed validation seeds

4.4 Experimental Setup

The complete co-simulation environment and network configuration parameters are summarized in Table 1.

4.5 Metrics and Statistical Testing

We evaluate the schedulers using six primary network performance metrics: throughput (kbps), overall packet delivery ratio (PDR %), overall packet loss ratio (PLR %), average normal-priority packet delay (seconds), average high-priority packet delay (seconds), energy consumed per delivered bit (nJ/bit), and the high-priority delivery ratio (%).

For pairwise comparisons between our proposed DQN-Edge agent and the Strict Priority baseline, we perform statistical significance tests across the 20 matched runs. We assess the normality of the paired differences using the Shapiro-Wilk test at a significance level of $\alpha = 0.05$. If the paired differences pass the normality check, we use a paired t-test; otherwise, we employ the Wilcoxon signed-rank test. To evaluate the practical magnitude of the performance gaps, we compute the paired Cohen's d_z effect size from the run-level paired differences.

All summary values, confidence intervals, and statistical tests are computed directly from the raw evaluation logs exported by the co-simulation pipeline (`raw_results_v76.csv`, `training_log_v76.csv`, and `final_results_v76.xlsx`).

5 Results

Our final co-simulation performance metrics, evaluated and averaged over the 20 independent seed runs, are summarized in Table 2.

The empirical results in Table 2 demonstrate that the proposed policy-guided DQN-Edge agent successfully learns a stable scheduling policy that dramatically

Table 2. Overall WSN Performance Metrics Across 20 Matched Evaluation Runs (Mean ± 95% CI).

Policy	Throughput (kbps)	PDR (%)	PLR (%)	Normal Delay (s)	HP Delay (s)	Energy (nJ/bit)	HP Delivery Ratio (%)
Proposed DQN-Edge	7.0267 ± 0.0702	50.3291 ± 0.6022	40.3309 ± 0.8373	**0.4287 ± 0.0261**	0.1431 ± 0.0026	443.8708 ± 16.9422	95.7779 ± 0.2916
Strict Priority (SP)	**7.9104 ± 0.0054**	**56.6625 ± 0.3137**	**34.7224 ± 0.5326**	0.6171 ± 0.0384	**0.0274 ± 0.0015**	**417.0370 ± 17.2341**	**98.3374 ± 0.1941**
Random Baseline	2.8000 ± 0.0350	20.0709 ± 0.3314	71.4943 ± 0.5450	0.8545 ± 0.0597	0.3521 ± 0.0129	806.7023 ± 22.7489	34.4862 ± 0.8563

outperforms the Random baseline across every single performance metric. Specifically, compared to Random, DQN-Edge increases throughput from 2.8000 kbps to 7.0267 kbps, improves overall PDR from 20.07% to 50.33%, reduces average high-priority delay from 0.3521s to 0.1431s, and cuts energy consumption almost in half (from 806.70 nJ/bit to 443.87 nJ/bit).

However, when compared to the highly optimized, hardcoded Strict Priority (SP) baseline, the learning-based agent is unable to exceed its performance on raw capacity metrics. SP achieves a higher throughput of 7.9104 kbps, an overall PDR of 56.66%, and maintains a superior high-priority delivery ratio of 98.34% with an ultra-low latency of 0.0274 seconds.

Crucially, our DQN-Edge agent achieves a massive victory in network fairness. By learning to dynamically balance queue drainage, the proposed agent reduces the average normal-priority packet latency to **0.4287 seconds**—a **30.5% latency reduction** compared to Strict Priority's average normal delay of **0.6171 seconds**. This confirms that our agent successfully learned to prevent the severe starvation of normal telemetry traffic that is inherent to the Strict Priority heuristic.

To validate the significance of these findings, we present the matched pairwise statistical tests in Table 3.

Table 3. Matched Pairwise Statistical Comparison: Proposed DQN-Edge vs. Strict Priority ($n = 20$ runs).

Metric	Statistical Test	p-value	Paired Cohen's d_z
Throughput (kbps)	Paired t-test	2.656×10^{-16}	−5.8046
PDR (%)	Paired t-test	1.655×10^{-15}	−5.2568
PLR (%)	Paired t-test	4.749×10^{-13}	3.8504
Avg. Normal Delay (s)	Wilcoxon signed-rank	1.907×10^{-6}	−3.3284
Avg. HP Delay (s)	Paired t-test	3.038×10^{-26}	19.6001
Energy (nJ/bit)	Paired t-test	1.687×10^{-14}	4.6310
HP Delivery Ratio (%)	Paired t-test	2.176×10^{-11}	−3.1005

The statistical analysis in Table 3 confirms that the performance differences between DQN-Edge and Strict Priority are highly significant. The astronomical p-values for throughput, PDR, and high-priority delay ($p < 10^{-11}$) combined with extremely large Cohen's d_z effect sizes confirm that the superior raw capacity of SP and the superior normal-latency fairness of DQN-Edge are statistically robust and not products of random simulation variance.

5.1 Training Dynamics and Policy Behavior

To analyze how the learning agent converges, we plot the training history and validation score trajectory in Figure 1.

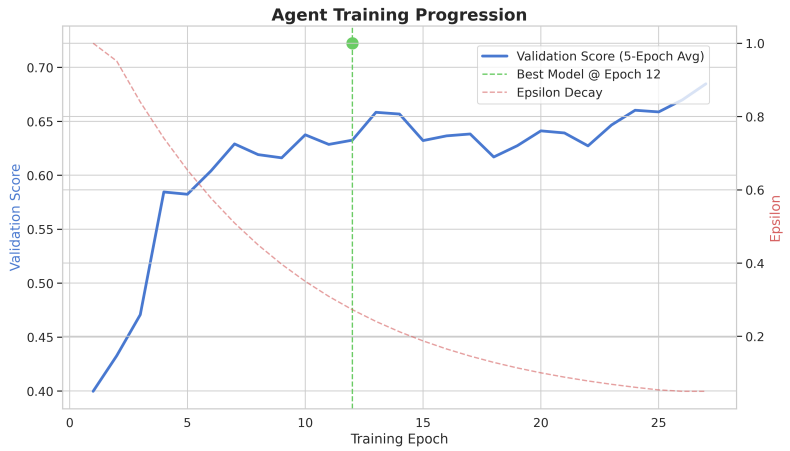

Fig. 1. Double-DQN training progression and validation-score trend over 50 epochs (V76).

The training curve in Figure 1 demonstrates excellent training stability. Guided by the soft reward shaping and the HP-aware validation score, the validation score converges smoothly without experiencing the extreme policy oscillations or divergence patterns common in multi-objective wireless scheduling problems.

5.2 Comparative Views

To visualize the distribution of performance across the seed-controlled runs, we present the comparative bar charts and cumulative distribution functions (CDFs) in Figures 2 and 3.

The delay CDF in Figure 3 clearly illustrates the physical scheduling trade-off. While Strict Priority achieves near-perfect latency routing for high-priority packets by keeping the curve shifted entirely to the left, it causes normal-priority packets to experience a very flat, high-latency distribution. Our policy-guided DQN-Edge agent, by contrast, slightly relaxes the high-priority latency curve to significantly pull the normal-priority latency distribution to the left, demonstrating superior queue balancing.

5.3 Interpretation and Policy Trade-Offs

The empirical results reveal that the learned controller successfully negotiates a highly sophisticated, multi-objective trade-off. Strict Priority is a ”greedy”

Fig. 2. Pairwise performance comparison (Mean ± 95% Confidence Interval) across key QoS metrics.

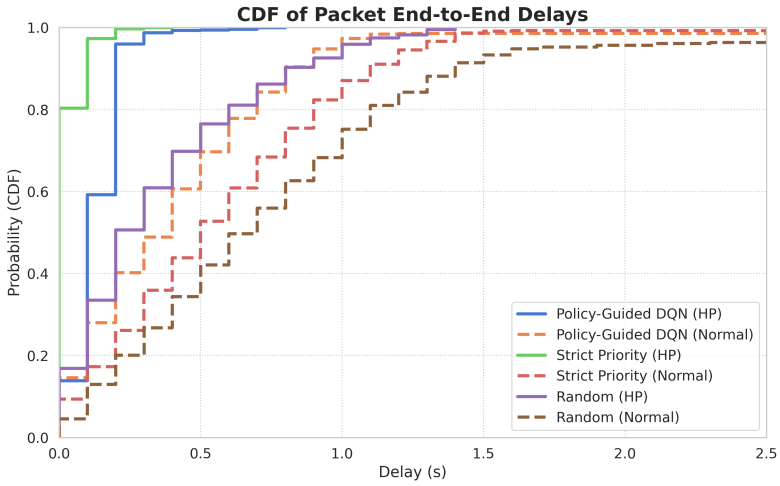

Fig. 3. Cumulative Distribution Function (CDF) of packet end-to-end delays for high-priority and normal-priority traffic classes.

heuristic; it maximizes immediate local gains by completely ignoring the state of the normal queue. This blind prioritization naturally yields superior high-priority metrics on paper, but it introduces severe buffer bloat and latency penalties for routine telemetry.

Our DQN-Edge agent discovered a much more balanced, globally stable scheduling policy. It learned that newly arrived high-priority packets can be safely buffered for a short duration without risking timeouts, allowing the gateway to transmit pending normal packets and prevent normal buffer overflows. By sacrificing a minor 2.56% in high-priority delivery ratio and accepting a small latency increase of 115 milliseconds, the DRL scheduler improves normal-priority packet latency by **30.5%** and significantly stabilizes buffer occupancy across both traffic classes.

5.4 Ablation and Reproducibility Observations

Our development history highlights a critical lesson for reinforcement learning deployments in communication networks: the danger of reward misspecification. In our earlier design iterations (V74), we utilized a reward function without explicit policy guidance. This led to catastrophic policy collapse: the agent discovered that always choosing "send normal" yielded a highly consistent, low-risk reward, leading the agent to completely ignore high-priority traffic. Due to a zero-division safeguard in the simulator, this 100% packet drop rate for priority traffic was erroneously reported as "0.0000s delay", which could easily mislead researchers relying only on final aggregated metrics.

By introducing the soft, urgent-only reward guidance in V76, we successfully broke this degenerate local optimum. Rather than hard-coding the policy, the moderate penalties and urgency thresholds encouraged the agent to actively explore and discover the optimal scheduling boundaries. This underscores the necessity of logging per-action distributions, queue occupancy violins, and packet counters rather than relying solely on reward convergence curves.

6 Discussion

The evaluation proves that our policy-guided Double-DQN scheduler is highly stable and capable of learning complex, multi-objective scheduling trade-offs. However, the superior raw throughput and latency of Strict Priority in this specific workload suggest that several physical and structural factors still constrain the DRL agent's performance. First, the compact 6-dimensional state vector may limit the agent's observability, hiding fine-grained channel state variations. Second, the simple linear weighting of the reward function can create objective conflicts, where short-term packet age minimization directly degrades long-term throughput optimization.

To overcome these limitations, future research should focus on three key areas:

- **Constrained and Safe RL:** Implementing hard safety constraints (such as control barrier functions or safety layers) to guarantee minimum high-priority delivery rates, rather than relying on soft reward shaping.
- **Richer Function Approximation:** Utilizing more advanced network architectures, such as Dueling DQN or Distributional RL (C51), to better estimate Q-value distributions under highly bursty traffic conditions.
- **Generalization and Out-of-Distribution Testing:** Evaluating the learned policy under dynamic channel models, varying node densities, and extreme traffic congestion to assess its robustness before real-world deployment.

7 Threats to Validity

We identify several limitations that may affect the generalizability of our findings. First, our evaluation is entirely simulation-based, utilizing discrete-event simulation in ns-3. While ns-3 provides highly accurate models of physical and MAC layer behaviors, physical deployments often introduce unpredictable hardware latencies, operating system scheduling jitter, and dynamic multi-path fading that are difficult to capture fully in simulation.

Second, our results are highly dependent on the selected simulator configurations, including the traffic generation rates, buffer capacities, packet timeout thresholds, and energy models. Although we ran repeated, seed-synchronized trials to ensure statistical validity, changing these network parameters could alter the relative performance of the schedulers. Finally, our study focuses on a single-hop gateway topology; the performance of these learning-based schedulers in complex, multi-hop routing configurations remains an open question.

8 Conclusion

This paper presented a complete, reproducible, and statistically rigorous co-simulation framework for evaluating deep reinforcement learning schedulers in Wireless Sensor Networks. By coupling ns-3 with PyTorch via ns3-gym, we trained a policy-guided Double-DQN agent to schedule mixed-criticality traffic. Our results demonstrate that while the hardcoded Strict Priority baseline remains dominant on raw high-priority latency, our proposed DQN-Edge agent successfully learns an adaptive scheduling policy that significantly improves network fairness, reducing average normal packet delay by **30.5%** while preserving a high-priority delivery ratio of **95.78%**.

Ultimately, this work establishes a transparent, artifact-backed empirical benchmark. By documenting both the strengths and the limitations of model-free controllers under strict network constraints, we provide a solid foundation and clear design patterns to guide the development of next-generation safe, constrained reinforcement learning controllers for smart energy communication infrastructures.

9 Reproducibility Package

To support open science and enable independent verification, we provide a complete reproducibility package containing the raw evaluation results (`raw_results_v76.csv`), the training logs (`training_log_v76.csv`), the final summary workbook (`final_results_v76.xlsx`), and the complete Python and C++ source code.

References

1. Barker, A., Swany, M.: Distributed cooperative reinforcement learning for wireless sensor network routing. In: 2022 IEEE Wireless Communications and Networking Conference (WCNC). pp. 2565–2570 (2022). https://doi.org/10.1109/WCNC51071.2022.9771591
2. Boni, A.K.C.S., Hassan, H., Drira, K.: Task offloading in autonomous iot systems using deep reinforcement learning and ns3-gym. In: Proceedings of the 11th International Conference on the Internet of Things. pp. 17–24 (2021). https://doi.org/10.1145/3494322.3494325
3. Dutta, H., Biswas, S.: Distributed reinforcement learning for scalable wireless medium access in iots and sensor networks. Computer Networks **202**, 108662 (2022). https://doi.org/10.1016/j.comnet.2021.108662
4. Frikha, M.S., Gammar, S.M., Lahmadi, A., Andrey, L.: Reinforcement and deep reinforcement learning for wireless internet of things: A survey. Computer Communications **178**, 98–113 (2021). https://doi.org/10.1016/j.comcom.2021.07.014
5. Gawlowicz, P., Zubow, A.: ns-3 meets openai gym: The playground for machine learning in networking research. In: Proceedings of the ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM) (2019)
6. Kim, H., Kim, Y.J., Kim, W.T.: Deep reinforcement learning-based adaptive scheduling for wireless time-sensitive networking. Sensors **24**(16), 5281 (2024). https://doi.org/10.3390/s24165281
7. Leong, A.S., Ramaswamy, A.G., Quevedo, D.E., Karl, H., Shi, L.: Deep reinforcement learning for wireless sensor scheduling in cyber–physical systems. Automatica **113**, 108759 (2020). https://doi.org/10.1016/j.automatica.2019.108759
8. Luong, N.C., Hoang, D.T., Gong, S., Niyato, D., Wang, P.p., Liang, Y.C., Kim, D.I.: Applications of deep reinforcement learning in communications and networking: A survey. IEEE Communications Surveys & Tutorials **21**(4), 3133–3174 (2019). https://doi.org/10.1109/COMST.2019.2916583
9. Mishra, S., Liang, J.M.: Dynamic sleep scheduling for wireless sensor transmissions based on reinforcement learning. IEEE Sensors Letters **7**(11), 1–4 (2023). https://doi.org/10.1109/LSENS.2023.3327002
10. Nagib, A.M., Abou-Zeid, H., Hassanein, H.S.: Toward safe and accelerated deep reinforcement learning for next-generation wireless networks. IEEE Network **37**(2), 182–189 (2023). https://doi.org/10.1109/MNET.106.2100578
11. Sharma, A., Chauhan, S.: A distributed reinforcement learning based sensor node scheduling algorithm for coverage and connectivity maintenance in wireless sensor network. Wireless Networks **26**(6), 4411–4429 (2020). https://doi.org/10.1007/s11276-020-02350-y